

Guia Básico de Sobrevivência: 1º Hackathon Híbrido UGB

Sumário: Guia Básico de Sobrevivência

Introdução: Leia antes de começar

- O Guia é uma bússola, não uma prisão.
- Antes de tudo se divirta aprendendo.
- Não, não é possível te ensinar a programar em 2 dias (mas este é um excelente primeiro passo).
- Sim, é muita coisa. Por que ninguém espera que você faça tudo em 48 horas.
- Trabalho em equipe: Dividir para conquistar.
- Defina prioridades.

Fase 1: Desenvolvendo a Ideia

- 1.1. O Público-Alvo: Existe alguém que realmente usaria o sistema?
- 1.2. Validação: A ideia é boa e viável para um fim de semana?
- 1.3. Levantamento de Requisitos: O que o sistema *precisa* fazer vs. O que seria *legal* fazer.

Fase 2: Escolhendo suas Armas (Tecnologia)

- 2.1. O Trio Básico: Front-end, Back-end e Banco de Dados.
- 2.2. Não reinvente a roda: O poder das Bibliotecas e Frameworks.

Fase 3: Por onde começar

- 3.1. Prototipação (Front-first): Rascunhando as telas e o fluxo do usuário. (Como usar Google AI Studio, Figma, Lovable, Excalidraw)
- 3.2. Modelagem (Back-first): Desenhando as tabelas do Banco de Dados.

Fase 4: Diário de Bordo (Git e GitHub)

- 4.1. O "GitHub Flow" para Hackathons
- 4.2. Padrão Convencional de Commits (Escrevendo commits legíveis)

Fase 5: O Básico de Orientação a Objetos (POO)

- 5.1. Classe
- 5.2. Atributos
- 5.3. Métodos
- 5.4. Objeto

Fase 6: Entendendo a Arquitetura (Como as peças se encaixam)

- 6.1. Arquitetura 1: O "Tudo em Um" (MVC Tradicional / Monolito)
 - Ideal para frameworks como Django, Laravel ou .NET MVC.
 - O padrão MVC (Model, View, Controller): Back e Front morando na mesma casa.
- 6.2. Arquitetura 2: Sistemas Separados (API + Front-end Independente)
 - Ideal para stacks como React/Vue no Front + Node/Java/Python no Back.
 - O que é uma API REST? (O garçom do seu sistema).
 - Dividindo o Back-end: *Controllers* (Portas de entrada), *Models* (Tabelas), *Services* (Regras de negócio) e *DTOs* (Os carteiros de dados).

Fase 7: Boas Práticas de Desenvolvimento (Evitando o Código Espaguete)

- 7.1. Arquivos com responsabilidades específicas: Não crie o arquivo "faz_tudo.js".
- 7.2. Métodos pequenos: Cada função faz apenas uma única coisa (e faz bem).
- 7.3. Nomes claros: Como nomear variáveis para você e seus companheiros entenderem.

Fase 8: A Lataria e o Pannel (Construindo o Visual)

- 8.1. Criando as telas e componentes.
- 8.2. Conectando a interface aos dados (seja via Template do MVC ou consumindo a API separada).

Fase 9: Os Bastidores (Infra e Segurança)

- 9.1. Conexão com o Banco de Dados.
- 9.2. Variáveis de Ambiente (.env).
- 9.3. Boas práticas de Segurança (Para quem tiver tempo!): Tokens JWT e restrição de acesso.

Fase 10: O Botão de Pânico (Rotas de Fuga e Apresentação)

- 10.1. Travou no Back-end? Como usar "Mocks" no Front-end e salvar a sua apresentação.
- 10.2. O histórico do Git como prova de esforço.

Introdução: Leia antes de começar

Bem-vindo(a) ao seu primeiro (ou mais um) Hackathon! Antes de começarmos, precisamos alinhar algumas ideias para que o seu final de semana seja incrível:

- **O Guia é uma bússola, não uma prisão:** Este documento não é uma regra absoluta. Outras arquiteturas, linguagens e formas de criar software são extremamente bem-vindas! Muito provavelmente, vocês não vão conseguir implementar tudo o que está escrito aqui devido ao tempo apertado. Levem este arquivo como um guia de entrada para dar uma noção geral do que fazer e por onde começar, e não como uma lista de tarefas obrigatória.
 - **Antes de tudo, divirta-se aprendendo:** O verdadeiro valor está no que você vai descobrir no processo. O objetivo principal deste evento não é criar um sistema impecável, mas tentar, errar, consertar e absorver conhecimentos que você nem sabia que precisava ter. Entregar algo simples e aprender no caminho é muito mais valioso do que tentar fazer algo perfeito e paralisar. É exatamente por isso que nossa avaliação é dividida em vários critérios (Pitch, UX, Inovação, etc.) — você não precisa ser um mestre da programação para pontuar.
 - **Não, não é possível te ensinar a programar em 2 dias:** Se você chegou aqui sabendo muito pouco, respire fundo e relaxe. Ninguém vira um engenheiro de software em um final de semana. No entanto, encare este evento como o melhor, mais rápido e mais intenso primeiro passo que você poderia dar na sua carreira.
 - **Sim, é muita coisa. E ninguém espera que você faça tudo em 48 horas:** Ao olhar para o tamanho do desafio ou para os próximos passos deste guia, pode bater um leve desespero. O sumário deste documento é apenas um mapa com várias rotas. Você e sua equipe decidem até onde conseguem caminhar. O foco é sobreviver e entregar, não gabaritar todas as necessidades.
 - **Trabalho em equipe (Dividir para conquistar):** Se todos tentarem fazer exatamente a mesma coisa ao mesmo tempo, o prazo vai acabar. Confiem uns nos outros e separem as tarefas: enquanto alguém rascunha as telas e o fluxo visual, outro pesquisa como fazer o código funcionar e um terceiro já começa a montar a apresentação final (Pitch).
 - **Defina prioridades:** Saiba diferenciar muito bem o que é o **MVP** (Produto Mínimo Viável) do que é um produto completo. O núcleo principal do seu sistema precisa funcionar para resolver o problema proposto. Animações bonitas, modo escuro e botões que piscam e funcionalidades complexas são enfeites que ficam para o final, apenas se sobra tempo.
-

Fase 1: Desenvolvendo a Ideia

Nenhum software bom salva uma ideia ruim ou mal planejada. Antes de escolher qual linguagem de programação usar ou criar o repositório, vocês precisam definir com clareza o que vão construir. Usem a noite de revelação do tema ou as primeiras horas do evento para debater os três pontos abaixo:

1.1. O Público-Alvo: Existe alguém que realmente usaria o sistema?

Toda boa solução de tecnologia nasce de um problema real. Quem é o seu usuário final? É um estudante universitário que não consegue organizar as horas complementares? É o dono de uma padaria de bairro que perde o controle do estoque no caderno?

- **Foque em nichos(se possível):** Evitem criar um aplicativo que serve para todo mundo. Quem tenta fazer tudo para todos, acaba não resolvendo o problema de ninguém. Escolham um nicho claro e foquem em solucionar a dor específica desse grupo.

1.2. Validação: A ideia é boa e viável para um fim de semana?

Lembrem-se: o relógio está correndo e vocês têm menos de 48 horas. Criar uma rede social do zero para competir com o Instagram é impossível. Criar um aplicativo simples de três telas que conecta alunos da faculdade para dividir carona é totalmente viável.

- **Se perguntem:** *"Nós temos o conhecimento técnico básico (ou a capacidade de pesquisar e aprender a tempo) para entregar a funcionalidade principal dessa ideia até domingo?"*. Se a resposta parecer não, simplifiquem a ideia, mas não necessariamente desistam dela.

1.3. Levantamento de Requisitos: O que o sistema precisa fazer vs. O que seria legal fazer.

Façam uma lista de tudo o que vocês esperam que o sistema tenha. Depois, sejam críticos e cortem essa lista para um tamanho realista pelo tempo.

- **O Essencial:** Imagine que a ideia de vocês seja desenvolver um software de gestão para uma oficina de carros. O sistema *precisa*, obrigatoriamente, permitir que você cadastre os carros, com as informações do dono do veículo, os problemas e orçamentos. Esse é o núcleo do projeto (o seu MVP). Sem isso, a solução simplesmente não funciona.
- **Seria Legal:** Um disparo automático de mensagem no WhatsApp avisando que o carro está pronto ou um dashboard com gráficos 3D sobre o faturamento da oficina são coisas incríveis. Mas, em um hackathon, isso é exagero. Deixem essas ideias anotadas para implementar apenas se o núcleo essencial já estiver construído e rodando sem grandes problemas.

Fase 2: Escolhendo Tecnologias

Vocês já sabem o que vão construir. Agora precisam decidir como vão construir.

Optem por aquilo que pelo menos uma pessoa da equipe já tenha alguma familiaridade. O relógio está correndo e gastar 12 horas apenas tentando instalar uma linguagem nova no computador pode ser frustrante. Porém, se o objetivo principal da sua equipe neste hackathon for puramente aprender algo novo do zero, sintam-se totalmente livres para experimentar aquela tecnologia que vocês sempre tiveram curiosidade. Só alinhem as expectativas!

2.1. O Trio Básico: Front-end, Back-end e Banco de Dados

Se você está nos primeiros períodos e esses nomes não parecem familiares, vamos simplificar. Pense no seu sistema como se fosse um restaurante:

- **Front-end (A Fachada e o Salão):** É tudo aquilo que o usuário vê, clica e interage. É a interface bonita do aplicativo ou do site. No nosso exemplo, é o cardápio, a decoração e as mesas.
 - *Armas comuns:* HTML, CSS, JavaScript, React, Vue. (Se o código travar, O Figma ou outras ferramentas podem resolver a parte visual para a apresentação).
- **Back-end (A Cozinha):** É o cérebro do sistema, onde a mágica real acontece longe dos olhos do cliente. É aqui que o pedido é recebido, os cálculos são feitos e a regra de negócio é aplicada.
 - *Armas comuns:* Node.js, Python, Java, C#, PHP.
- **Banco de Dados (A Despensa e o Livro de Registros):** É a memória do sistema. É onde as informações ficam guardadas para não sumirem quando o usuário atualiza a página. Se alguém faz um cadastro, é para cá que os dados vão.
 - *Armas comuns:* MySQL, PostgreSQL, MongoDB, SQLite, SQL Server. (Lembrando: em um hackathon, se o banco não der certo, salvar os dados na memória temporária do código ou usar mocks!).

2.2. Não reinvente a roda: O poder das Bibliotecas e Frameworks

O tempo é o seu recurso mais escasso. Imagine que você precisa construir um carro. Você não vai minerar o ferro, derreter, moldar os parafusos e fabricar a borracha do pneu. Você pega peças prontas, junta tudo e foca em fazer o motor funcionar.

- **Bibliotecas e Frameworks** são basicamente códigos que outros programadores já escreveram e disponibilizaram de graça na internet para resolver problemas comuns. Eles são os seus atalhos.
- Precisa de botões bonitos e um layout responsivo (que funciona no celular) sem ter que escrever mil linhas de CSS? Use *Bootstrap* ou *Tailwind*.
- Vai criar uma API no Back-end? Não escreva o servidor do zero, use *Express* (se for Node), *Django* ou *FastAPI* (se for Python), *Ecossistema .NET* (se for C#).

Frameworks são como ecossistemas completos. Usem e abusem deles. O mercado de trabalho real funciona exatamente assim: ninguém perde tempo reinventando o que já está pronto e testado.

Fase 3: Por onde começar

Ideia validada e requisitos definidos. Para destravar o projeto, as equipes de software geralmente escolhem um de dois caminhos para dar o pontapé inicial. Vocês podem escolher o que faz mais sentido para a equipe, ou até mesmo se dividir em duplas para fazer os dois ao mesmo tempo.

3.1. Prototipação: Rascunhando as telas e o fluxo do usuário.

Se a equipe é mais visual ou se a interface é a parte mais importante do aplicativo de vocês, comecem pelo Front-end visual. Mas atenção: **ainda não é hora de programar**.

- Se ele clicar no botão "Salvar", para qual tela ele é redirecionado? Aparece um aviso de sucesso?
- **Por que isso ajuda?** Quando vocês têm o desenho das telas pronto, fica infinitamente mais fácil para o restante da equipe saber exatamente quais dados o Back-end vai precisar fornecer. Se na tela tem um campo "Idade do Cliente", o banco de dados obrigatoriamente vai precisar de uma coluna para salvar a idade. O Front-end serve como um mapa para o Back-end.

3.1. Prototipação (Front-first): Rascunhando as telas e o fluxo do usuário

A maneira mais fácil de uma equipe entender o que está construindo é começar pelo visual. Quando todo mundo consegue "ver" o aplicativo, as ideias se alinham instantaneamente. Não pulem direto para o código; gastem as primeiras horas rascunhando.

As Ferramentas de Prototipação (Escolha a sua arma):

- **Excalidraw:** É o quadro branco virtual mais rápido da internet. Perfeito para rascunhos feios, caixas quadradas e setas ligando uma tela na outra. Excelente para sexta-feira à noite.
- **Figma:** A ferramenta padrão da indústria para design de interfaces. Usem se alguém da equipe já souber mexer, mas cuidado com a armadilha do perfeccionismo! O botão não precisa ter a sombra perfeita agora. Pode ser complicada de mexer no começo
- **Lovable (ou similares como v0):** O "hack" moderno dos hackathons. São IAs onde você digita o que quer (ex: *"Crie uma tela de login moderna com tema escuro"*) e elas geram a interface e até o código Front-end inicial para você.
- **Google AI Studio:** Excelente parceiro de *brainstorming*. Vocês podem descrever a ideia do aplicativo lá e pedir para a IA listar quais telas seriam necessárias ou qual seria o fluxo lógico ideal para o usuário não se perder.

Entendendo o Fluxo do Usuário (O Caminho pelas Telas): Antes de escrever a primeira linha de código desenhem. Quais serão as telas do sistema? Uma tela de Login, uma tela Principal (Dashboard) e uma tela de Cadastro? **O Fluxo do Usuário:** Pensem como o usuário vai navegar.

Vocês precisam pensar em cada passo, como se estivessem guiando alguém com os olhos vendados:

1. **O Ponto de Partida:** O usuário abre o app e vê a Tela de Login. Ele digita os dados.
2. **A Ação:** Ele clica no botão "Entrar".
3. **O Caminho Triste (Erros):** E se ele digitar a senha errada? A tela recarrega? Aparece um texto vermelho piscando dizendo "Senha Inválida"? Onde esse aviso aparece?
4. **O Caminho Feliz (Sucesso):** A senha está certa. Para qual tela ele é redirecionado? O Dashboard principal? Aparece um aviso verde de "Bem-vindo"?

5. **Os Becos sem Saída:** Se ele clicar em "Esqueci minha senha", tem uma tela pra isso ou o botão é falso (mockado)?

Dica: Evitem criar buracos sem voltas no aplicativo. Toda tela precisa ter um botão de "Voltar" ou um menu para o usuário conseguir retornar ao início sem ter que fechar o sistema.

Por que isso ajuda TANTO o resto da equipe? (O Mapa): Quando vocês têm o desenho das telas e o fluxo prontos, a vida de quem vai programar o Back-end e o Banco de Dados fica infinitamente mais fácil.

O Front-end funciona como um contrato visual. Se na tela de cadastro vocês desenharam os campos "*Nome*", "*E-mail*", "*Senha*" e "*Idade do Paciente*", o colega que está configurando o Banco de Dados já sabe automaticamente que a tabela dele precisará, obrigatoriamente, de colunas exatas para essas 4 informações. A tela diz ao banco de dados o que ele precisa guardar.

3.2. Modelagem (Back-first): Desenhando as tabelas do Banco de Dados.

Se o projeto de vocês é focado em processar muita informação e regras de negócio complexas, o melhor caminho é começar do banco para o front, estruturando como a informação será armazenada.

- **Pense em planilhas do Excel:** Um banco de dados nada mais é do que um conjunto de tabelas muito inteligentes. Que tipo de informações o sistema de vocês vai guardar?
- **Definindo as Entidades:** Façam uma lista. Vocês vão precisar de uma tabela de **Usuários** (com as colunas *Nome*, *E-mail* e *Senha*)? Vão precisar de uma tabela de **Produtos** (com *Nome*, *Preço* e *Quantidade*)?
- **Por que isso ajuda?** A estrutura do banco de dados é a base do sistema. Quando vocês definem logo de cara quais dados existem e como eles se relacionam (exemplo: Um "Usuário" pode comprar vários "Produtos"), construir a lógica do Back-end depois vira apenas um trabalho de implementar regras de negocio e criar as rotas para salvar ou buscar essas informações que já estão bem definidas.

Fase 4: Diário de Bordo (Git e GitHub)

Programar em equipe sem usar o Git é como tentar escrever um livro a quatro mãos usando o mesmo caderno físico. Para que duas ou mais pessoas consigam escrever código ao mesmo tempo no mesmo projeto, vocês usarão o Git e o GitHub.

4.1. O "GitHub Flow" para Hackathons

O GitHub Flow é uma forma organizada de trabalhar com o código sem quebrar o que já está funcionando. A regra é simples:

- **A Branch `main` (O Código Principal):** A branch principal (`main`) é a versão oficial do seu projeto. É a versão que vai ser apresentada para a banca. **NUNCA programe direto na `main` e nunca jogue código quebrado nela.**
- **Branches de Feature (O seu rascunho seguro):** O João vai fazer a tela de Login? Ele não faz na `main`. Ele cria uma cópia segura chamada *branch* `main` (ex: `feature/tela-login`). A Maria vai fazer a conexão com o banco? Ela cria a branch `feature/banco-dados`.
- **O Merge (Juntando as branches):** O João terminou a tela e ela está funcionando perfeitamente? Agora sim ele faz um *Pull Request* (ou um *Merge*) para juntar a `feature/tela-login` dentro da `main`.

Se o código do João explodir e der tudo errado na branch dele, a `main` continua intacta e a apresentação da equipe está salva!

4.2. Padrão Convencional de Commits (Escrevendo commits legíveis)

Lembrem-se: Vão auditar o GitHub de vocês no domingo à noite. Se abrirem o histórico e virem mensagens como "`arrumei a parada`", "`agora vai`" ou "`asdfghj`", a impressão técnica da equipe cai drasticamente.

Use os Commits Convencionais. É um padrão de mercado que usa padrões no começo da mensagem para explicar rapidamente o que aquele código faz.

Acostumem-se a usar estes prefixos nas mensagens de vocês:

- **feat:** (Feature) -> Quando você adiciona uma funcionalidade nova.
 - Ex: `feat: cria tela de cadastro de usuários`
- **fix:** (Correção) -> Quando você conserta um bug no código.
 - Ex: `fix: resolve botão de salvar que não estava clicando`
- **docs:** (Documentação) -> Quando você altera o README ou adiciona comentários.
 - Ex: `docs: adiciona instruções de como rodar o projeto`
- **chore:** (Manutenção) -> Quando você instala uma biblioteca nova ou configura algo no sistema, mas não é uma funcionalidade para o usuário.
 - Ex: `chore: instala biblioteca axios para conectar com a API`
- **style:** (Estilo visual) -> Quando você só mexe em CSS, cores e alinhamentos.
 - Ex: `style: centraliza o título e muda a cor de fundo para azul`

No domingo, quando os avaliadores baterem o olho no histórico do GitHub de vocês, eles vão ver uma linha do tempo profissional, limpa e perfeitamente documentada. Isso mostra maturidade, organização e ganha muitos pontos de Execução Técnica.

Fase 5: O Básico de Orientação a Objetos (POO)

Antes de falarmos sobre como as peças do sistema se encaixam, precisamos entender como o código é escrito na maioria das linguagens modernas (como Java, C#, Python e até JavaScript). Vocês vão ouvir muito as palavras "Classe" e "Objeto". Se isso é novo para você, aqui vai a explicação rápida:

- **A Classe (O Molde/A Planta):** Pense na Classe como a planta arquitetônica de uma casa. Ela não é a casa real onde você pode entrar; ela é apenas o desenho dizendo que a casa deve ter portas, janelas e uma cor. No código, uma classe define o formato de algo (ex: a classe `Usuario` ou a classe `Produto`).
 - **Atributos (As Variáveis):** São as características do seu molde. A grosso modo, são as variáveis que ficam guardadas lá dentro. A nossa classe `Usuario` tem atributos como `nome`, `email` e `idade`.
 - **Métodos (As Funções/Ações):** São as ações que esse molde pode executar. O nosso `Usuario` pode ter um método chamado `fazerLogin()` ou `atualizarSenha()`.
 - **O Objeto (A Casa Pronta):** Quando o sistema está rodando e o usuário João faz um cadastro, o código usa aquela planta (a Classe) para construir uma casa de verdade na memória (o Objeto). A partir desse momento, você tem um objeto real e pode chamar as variáveis e funções específicas do João chamando algo como: `joao.fazerLogin()`.
-

Fase 6: Entendendo a Arquitetura (Como as peças se encaixam)

Agora que vocês escolheram as tecnologias, como organizar o código? Existem dois caminhos principais no mercado (e no nosso hackathon). Escolham o que fizer mais sentido para a equipe:

6.1. Arquitetura 1: O "Tudo em Um" (MVC Tradicional / Monolito)

Esta é a arquitetura clássica. Aqui, o Front-end e o Back-end moram na mesma casa, dividem a mesma conta de luz e rodam no mesmo servidor.

- **Ideal para:** Equipes que escolheram frameworks robustos como *Django* (Python), *Laravel* (PHP) ou *C# MVC* (.NET).
- **Como funciona o padrão MVC (Model, View, Controller):**
 - **Model (O Banco de Dados):** É o código que representa suas tabelas. Se você tem usuários, o Model dita que eles têm "nome, email e senha".
 - **View (A Tela):** É o HTML/CSS que o usuário vê. A diferença é que o próprio Back-end "costura" os dados do banco de dados direto no HTML antes de mandar para o navegador.

- **Controller (O Maestro):** É quem recebe o clique do usuário, pede a informação para o *Model* e entrega a resposta montada na *View*. Tudo acontece no mesmo lugar.

6.2. Arquitetura 2: Sistemas Separados (API + Front-end Independente)

Esta é a arquitetura mais usada nas empresas modernas. O Front-end e o Back-end moram em cidades diferentes e só conversam através da internet.

- **Ideal para:** Equipes que dividiram o trabalho, usando *React / Vue / Angular* de um lado (Front) e *Node.js / C# / Python* do outro (Back).

O que é uma API REST? (O Garçom do seu sistema) Se os sistemas estão separados, como eles conversam? Através da API. Imagine que o Front-end é o cliente sentado na mesa lendo o cardápio. O Back-end é a cozinha. O cliente não pode entrar na cozinha e pegar a comida. Ele precisa chamar o garçom (a API). O Front-end faz um "pedido" (uma requisição HTTP), a API leva até a cozinha (Back-end), a cozinha prepara a resposta (geralmente em um formato de texto chamado JSON) e a API devolve para a mesa.

Dividindo o Back-end (Para o código não virar bagunça): Se a cozinha (Back-end) for desorganizada, o pedido atrasa. Por isso, dividimos as responsabilidades em pastas e arquivos específicos:

1. **Controllers (As Portas de Entrada / O Recepcionista):** É o primeiro lugar onde o pedido da API chega. O Controller não pensa muito; ele só confere se o pedido faz sentido e manda para a pessoa certa resolver.
2. **Services (As Regras de Negócio / O Cozinheiro):** É aqui que a inteligência do sistema mora. Vai calcular um desconto? Validar se o usuário tem saldo? Enviar um e-mail? É o Service quem faz o trabalho pesado.
3. **Models (O Formato das Tabelas):** Assim como no MVC, são as classes que representam exatamente como as colunas estão desenhadas lá no Banco de Dados.
4. **DAOs ou Repositories (O Estoquista):** O cozinheiro (Service) não vai até a despensa pegar os ingredientes. Ele pede ao Repository. O Repository é a **única** parte do seu código que tem permissão para conversar diretamente com o Banco de Dados (fazer os *SELECTs*, *INSERTs*, etc.).
5. **DTOs - Data Transfer Objects (Os Carteiros/Entregadores):** Servem para filtrar o que entra e o que sai. Se a sua API vai devolver os dados de um usuário para o Front-end, você não pode enviar a senha dele junto por engano. O DTO cria um "pacote seguro" contendo só o Nome e o E-mail e entrega para o garçom levar até a mesa.

(Dica: Em um hackathon, se o tempo estiver acabando, às vezes não é viável separar tudo tão bem assim, ainda mais na primeira experiência, nesse caso foque em entregar. Não é o ideal para o mercado, mas para esse caso, a regra é fazer funcionar!)

Fase 7: Boas Práticas de Desenvolvimento (Evitando o Código Espaguete)

A tentação de escrever o código de qualquer jeito só para ver funcionando eventualmente vai aparecer. O problema é que o código espaguete é impossível de ler e arrumar. Um bug bobo em um código bagunçado pode custar 4 horas da sua equipe.

Para evitar os problemas e garantir que os mentores (e vocês mesmos) consigam ler o que foi feito, sigam estas três regras:

7.1. Arquivos com responsabilidades específicas: Não crie o arquivo "faz_tudo.js"

Se o seu arquivo principal (`index.js`, `App.js`, ou `main.py`) está chegando a 500 linhas no sábado à tarde. Vocês estão colocando o martelo, os pregos, o sanduíche e a planta da casa na mesma gaveta da caixa de ferramentas.

- **Dividir para achar os erros:** Separem as coisas fisicamente. Crie uma pasta só para as rotas, um arquivo só para conectar no banco de dados e outro só para as funções de usuário. Se o login quebrar, todo mundo da equipe sabe exatamente em qual arquivo procurar o problema, sem precisar rolar infinitamente a tela.

7.2. Métodos pequenos: Cada função faz apenas uma única coisa (e faz bem)

Um dos maiores erros de quem está começando é criar uma função gigante chamada `cadastrar_usuario()` que faz de tudo: ela confere se o e-mail tem "@", verifica se a senha é forte, salva no banco de dados, envia um e-mail de boas-vindas e ainda limpa a tela.

- **A Regra da Única Coisa:** Se a sua função tem muitas linhas, ela provavelmente está fazendo muito trabalho. Quebre-a em funções menores.
- **O benefício prático:** Se o envio de e-mail falhar, o sistema inteiro não quebra. Fica muito mais fácil isolar o erro na função `enviarEmailBoasVindas()` do que caçar o bug no meio de um bloco gigantesco de código.

7.3. Nomes claros: Como nomear variáveis para você e seus companheiros entenderem

Esqueça a preguiça. Programar não é sobre escrever para a máquina entender, é sobre escrever para **humanos** (seus colegas de equipe) lerem.

Você pode até achar que `let x = 10;` e `const d = dados;` fazem sentido no momento da escrita, mas no futuro nem você vai lembrar o que esse "x" significa.

- **Fuja de nomes genéricos:** Variáveis chamadas `teste`, `dados2`, `arr`, `info` ou `a`, `b`, `c` são proibidas.
 - **Seja descritivo (mesmo que fique longo):** Não tenha medo de escrever. Uma variável chamada `listaDeUsuariosAtivos` ou uma função chamada `calcularDescontoDoCarrinho()`. Qualquer membro da equipe que bater o olho nisso vai entender o que o código faz sem precisar perguntar para você.
-

Fase 8: Construindo o Visual

O Front-end é a vitrine do seu projeto. A banca avaliadora (e os usuários) não vão ver o seu banco de dados super otimizado, eles vão ver a tela. Uma tela bem montada, mesmo que simples, passa muita credibilidade.

8.1. Criando as telas e componentes

A regra número um do Front-end em um hackathon é: não perca tempo consturindo CSS puro se você não precisa. Use bibliotecas visuais: Escolha um *framework* ou biblioteca de componentes prontos (*Bootstrap*, *Tailwind CSS*, *shadcnUI*). Eles já te dão botões bonitos, formulários alinhados e menus que funcionam no celular (responsivos) com pouquíssimo esforço.

- **Pense em Componentes (Lego):** Se você está usando React, Vue ou Angular, não escreva a tela inteira em um arquivo só. Quebre a interface em pequenos blocos de montar.
 - O botão de "Salvar" vai ser usado em 5 telas diferentes? Crie um componente `<BotaoSalvar />` ou importe de uma biblioteca como *ShadcnUI* e reuse-o. Se no domingo vocês decidirem que o botão deve ser verde em vez de azul, você muda em um só lugar e o sistema inteiro se atualiza.
 - Isso também permite que duas pessoas trabalhem no Front-end ao mesmo tempo: uma faz o componente do *Header* (cabeçalho) e a outra faz o *Card* de produtos.

8.2. Conectando a interface aos dados

Ter uma tela bonita com dados mockados à mão no código é só o primeiro passo. A magia acontece quando o painel mostra dados reais. Como fazer isso depende da arquitetura que vocês escolheram na Fase 4:

- **Se vocês escolheram o MVC (Tudo em Um):** A conexão é direta. O seu Back-end (em Python, PHP, C#) vai pegar as informações do Banco de Dados e "injetar" diretamente no seu arquivo HTML antes de mandar para a tela do usuário. É um processo mais rápido de configurar no início, perfeito para quem quer entregar rápido. O usuário clica em algo, a página recarrega e traz a tela nova com os dados atualizados.

- **Se vocês separaram o Front e o Back (Consumindo a API):** Aqui a tela é independente. O seu Front-end (ex: React ou JavaScript puro) vai usar funções nativas como o `fetch()` ou bibliotecas como o `axios` para "ligar" para o garçom (a API).
 - **Atenção à assincronicidade:** A internet não é instantânea. O Front-end vai pedir os dados e ficar esperando. Lembre-se de colocar um ícone de "Carregando..." (Loading) na tela enquanto a resposta do Back-end não chega. Isso ganha muitos pontos de Experiência do Usuário (UX) com a banca!
 - **Atenção aos nomes das variáveis (O idioma da API):** Quando o Front-end vai enviar um dado para o Back-end (ou quando vai receber dados dele), os dois precisam falar exatamente o mesmo idioma. Se o seu Back-end está esperando receber um dado chamado `emailUsuario`, o Front-end não pode enviar um pacote escrito `email_user` ou `EmailUsuario` (com E maiúsculo). A programação é extremamente sensível a isso. Um único caractere diferente ou uma letra maiúscula no lugar errado faz o Back-end ignorar a informação, e o sistema quebra sem avisar o motivo. **Dica:** O programador do Front e o programador do Back precisam combinar e anotar os exatos nomes das variáveis que vão trafegar entre eles!
 - **A Dica de Sobrevivência (Mock First):** Antes de tentar conectar o Front-end com a API real, crie um arquivo falso no seu Front-end (`dadosFalsos.json`). Faça a sua tela ler esse arquivo primeiro. Quando a tela estiver perfeita exibindo os dados falsos, aí sim você troca o "endereço" para a API real. Se o Back-end explodir no domingo, você simplesmente volta a ler o arquivo falso e salva a apresentação!
-

Fase 9: Os Bastidores (Infra e Segurança)

Se o Front-end é a vitrine e o Back-end é a cozinha, a infraestrutura é a rede elétrica e o encanamento. O usuário não vê, mas se der problema, o restaurante inteiro para.

9.1. Conexão com o Banco de Dados e Hospedagem

Fazer o seu código conversar com o banco de dados costuma ser o primeiro grande desafio técnico. Vocês têm dois caminhos para isso, e ambos são válidos:

- **Opção 1: Banco Local (O mais rápido para começar):** Rodar o banco de dados direto na máquina de quem está programando o Back-end (usando MySQL, Postgres, XAMPP, etc.). É ótimo porque não depende de internet e é rápido de configurar. O lado ruim é que os outros membros da equipe não conseguem acessar o mesmo banco.
- **Opção 2: Banco na Nuvem:** Usar serviços gratuitos como *Supabase*, *Neon*, *MongoDB Atlas* ou *Firebase*. Assim, todos da equipe se conectam ao mesmo banco via internet.

- **A Regra de Ouro da Apresentação:** Vocês **NÃO** são obrigados a hospedar o sistema na internet (fazer *deploy*, usar Docker, configurar servidores). Rodar o projeto no `localhost` do seu computador está perfeito!
- **Medo de dar branco no palco? Usem Slides!** Para a apresentação de segunda-feira, se vocês tiverem medo do sistema travar ao vivo, é totalmente permitido tirar *prints* (fotos) ou gravar um vídeo curto da tela do sistema funcionando e colocar nos slides. A nossa equipe de mentores já terá feito a auditoria no GitHub de vocês no domingo para comprovar que o código existe e foi feito por vocês. O palco é para vender a ideia.

9.2. O Segredo do Cofre: Variáveis de Ambiente (`.env`)

NUNCA, em hipótese alguma, escrevam senhas de banco de dados, chaves de API ou tokens secretos diretamente no meio do código. Se vocês fizerem isso e enviarem para o GitHub (que é público), qualquer pessoa no mundo terá acesso ao sistema de vocês.

- **Arquivo `.env`:** Crie um arquivo chamado exatamente `.env` na raiz do seu projeto. Coloque todas as suas senhas e links secretos lá dentro (ex: `DATABASE_URL=senha_super_secreta`). O seu código vai ler essas variáveis puxando desse arquivo. **adicione o arquivo `.env` no seu `.gitignore`.** Isso faz com que tudo que está nas pastas subam menos arquivos que os caminhos estejam no `gitignore`
- **O `.env.example`:** Como a sua equipe vai saber qual é a senha se o arquivo não vai para o GitHub? Crie um arquivo chamado `.env.example` com o nome das variáveis, mas com valores falsos (ex: `DATABASE_URL=coloque_a_senha_aqui`). Esse arquivo sim vai para o Git, servindo de modelo para os seus colegas substituírem os dados.

9.3. Boas práticas de Segurança: Tokens JWT (Apenas se sobrar tempo!)

Lembram da regra de prioridades. Se a equipe estiver rápida e quiser impressionar os mentores na auditoria técnica, implementem um sistema de autenticação real.

- **O que é o JWT (JSON Web Token)?** Pense nele como a pulseira da área VIP de uma balada.
 - **Como funciona:** Quando o usuário faz o login e acerta a senha, o seu Back-end não precisa ficar perguntando a senha dele toda hora. O Back-end gera um token (a pulseira VIP) e entrega para o Front-end.
 - **Restrição de Acesso:** A partir desse momento, toda vez que o Front-end quiser buscar uma informação sensível (como o saldo da conta), ele mostra o token para a API. A API olha o token, verifica se é válido, se não está vencido e confirma: *"Ok, ele tem a pulseira, pode liberar os dados"*. Se o usuário tentar acessar uma rota de "Administrador" com um token de "Cliente comum", o Back-end barra a entrada.
-

Fase 10: O Botão de Pânico (Rotas de Fuga e Apresentação)

O banco de dados corrompeu, a API parou de responder e o servidor local simplesmente se recusa a ligar. Bateu o desespero? Respire fundo e aperte o Botão de Pânico.

O importante é ter algo para mostrar.

10.1. Travou no Back-end? Como usar "Mocks" no Front-end e salvar a sua apresentação

A banca de professores que vai assistir ao seu Pitch na segunda-feira quer ver a sua ideia ganhando vida. Se o motor do carro (Back-end) fundiu, a lataria (Front-end) ainda precisa estar bonita para a foto.

- **O que são Mocks?** São dados falsos, criados manualmente, para simular que o sistema está funcionando. É como um cenário de filme: a fachada da casa é perfeita, mas atrás é só madeira e papelão.
- **Como aplicar a Rota de Fuga:**
 - Desconecte o Front-end da sua API quebrada.
 - Crie variáveis fixas no seu código (ex: `const saldoUsuario = 150.00;`).
 - Crie listas falsas (`arrays`) para simular o que viria do banco de dados (ex: uma lista de 3 pacientes já cadastrados na mão).
 - Faça as suas telas lerem esses dados falsos.
- **O Resultado:** Quando você for apresentar, vai clicar nos botões, as telas vão mudar, os nomes falsos vão aparecer e a banca vai entender perfeitamente como o sistema deveria funcionar na vida real.